Testing

Evidence, Guarantees, and Risk in Software Systems

FinTech 512 — Software Engineering for Financial Technology

Duke

Where Testing Fits in the Course

From Decisions to Evidence

So far in this course, we have focused on:

- Requirements — deciding *what should happen*
- Design — deciding *where behavior lives*
- Patterns & principles — deciding *how decisions evolve safely*

Testing answers a different question:

How do we know those decisions still hold?

Why Testing Matters in Financial Systems

Financial Software Amplifies Mistakes

- Errors execute at machine speed
- Automation removes human checkpoints
- Small defects can trigger **large financial impact**

In Finance:

- Money moves instantly
- Failures are often **irreversible**
- Trust, once lost, is hard to recover

“Undo” and “extra life” don’t always exist.

What Testing Actually Does

Testing Makes Expectations Explicit

- Encodes expected behavior
- Freezes assumptions into executable form
- Defines what "correct" means *right now*

Testing Provides Evidence

- Evidence that behavior matches expectations
- Evidence that changes did not silently break guarantees
- Evidence that the system behaves *as decided*

Testing Does *Not* Prove Correctness

- **Passing tests ≠ correctness**
- **Failing tests = evidence of a problem**

Theoretically impossible to prove software is defect-free.

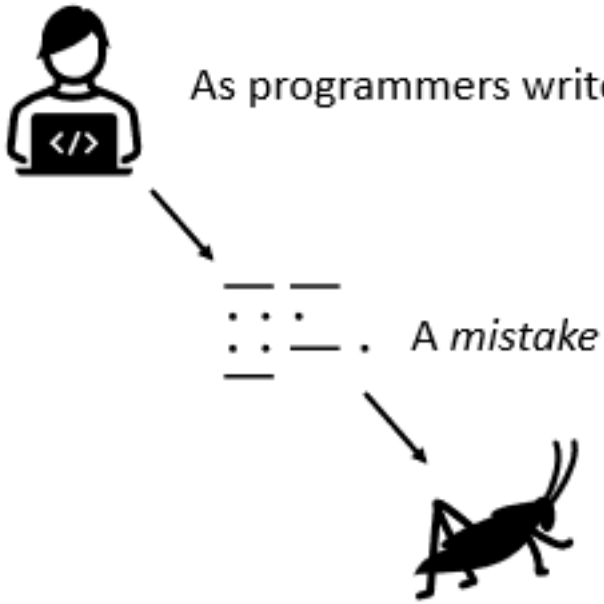
Software Testing

Software testing consists of the *dynamic* verification that a program provides *expected* behaviors on a *finite* set of test cases, suitably *selected* from the usually infinite execution domain.

Dynamic: We execute the program on select inputs
Expected: We observe the expected outputs of the program and decide if the test was successful or not.
Finite: Even the simplest of programs can have so many input values that exhaustive testing is infeasible. As such, we can only perform testing on a subset of the possible tests.
Selected: Selecting different test cases can have different levels of effectiveness. As such, we must seek to create the correct set of test cases to evaluate our programs effectively (minimize time and effort).

Software Faults, Errors & Failures

- Software Fault : A static defect in the software
- Software Failure : External, incorrect behavior with respect to the requirements or other description of the expected behavior
- Software Error : An incorrect internal state that is the manifestation of some fault



As programmers write code, they will invariably make *mistakes*

A *mistake* embodies itself as a *fault* (*defect*) in the code

A *failure* is observed when the *fault* is made visible. Otherwise, a *fault* remains *latent* until it is observed – which may never occur.

A Concrete Example

Fault: Should start searching at 0, not 1

```
public static int numZero (int [ ] arr)
{  // Effects: If arr is null throw NullPointerException
   // else return the number of occurrences of 0 in arr
   int count = 0;
   for (int i = 1; i < arr.length; i++)
   {
      if (arr [ i ] == 0)
      {
         count++;
      }
   }
   return count;
}
```

Test 1
[2, 7, 0]
Expected: 1
Actual: 1

Error: i is 1, not 0, on the first iteration
Failure: none

Test 2
[0, 2, 7]
Expected: 1
Actual: 0

Error: i is 1, not 0
Error propagates to the variable count
Failure: count is 0 at the return statement

Testing as Evidence, Not Assurance

What Testing Can Tell Us

- A failure indicates the presence of a fault
- A passing test indicates behavior matched expectations **for that case**

What Testing Cannot Tell Us

- That no faults exist
- That all behaviors are correct
- That future executions will behave the same

Key Reality

- **Absence of failures is not evidence of correctness.**
- **Absence of tests is absence of evidence.**

Validation & Verification

Verification

Did we build the system correctly?

- Conformance to specifications
- Contracts, invariants, expected behavior
- "Are we doing what we said we would do?"

Validation

Did we build the correct system?

- User needs
- Business intent
- Regulatory and domain fit

A system can be perfectly verified and still catastrophically wrong.

Formal Definitions (SWEBOK, IEEE Std. 1012:2016):

- **Validation**: The process of evaluating software at the end of software development to ensure compliance with intended usage
- **Verification**: The process of determining whether the products of a given phase of the software development process fulfill the requirements established during the previous phase

Testing as Intentional Evaluation

Testing Is About Asking the Right Questions

- Testing is not a single activity or technique
- Different tests exist to answer **different questions**
- Each test reflects:
  - a **risk**
  - an **assumption**
  - a **decision** about what matters

What did this test tell us?

Sources of Software Problems

Software failures rarely come from a single mistake.
They emerge from problems across **people, processes, and systems**.

Requirements

- Ambiguous, incomplete, or incorrect assumptions

Design

- Poor separation of concerns
- Missing failure behavior

Implementation

- Logic errors
- Boundary and edge cases

Tools & Configuration

- Build, deployment, and environment mismatches

Evolution

- Changes that invalidate prior assumptions

Human Interaction

- Misunderstanding requirements
- Misuse of tools
- Operational mistakes

Inadequate or Misguided Testing

- Missing scenarios
- Testing the wrong things

Automation & AI Assistance

- Generated code may be:
  - syntactically valid but semantically wrong
  - confident but incorrect
  - inconsistent across contexts
- Errors may be:
  - harder to reason about
  - harder to attribute
  - repeated systematically

Tactical Test Objectives

Why Does Any Given Test Exist?

Every test should exist for a **specific reason**.
Tests are written to answer questions and reduce particular risks.

Common Test Objectives

- **Verify required behavior**
  - Does the system do what was specified?
- **Detect regressions**
  - Did a change break something that used to work?
- **Explore edge and failure cases**
  - What happens at boundaries or under invalid conditions?
- **Increase confidence before change or release**
  - Do we have enough evidence to proceed?

Constraints Are Real

- Time is limited
- Budgets are finite
- Exhaustive testing is impossible

Testing is about prioritizing risk, not eliminating it.

Black-Box vs White Box Testing

Two Complementary Perspectives

Black-Box Testing

Focus: *What the system does*

- Based on requirements and user expectations
- Inputs → outputs → observed behavior
- Internal implementation is irrelevant

Typical questions:

- Does this feature behave as specified?
- Are edge cases handled correctly?
- What happens with invalid input?

White-Box Testing

Focus: *How the system works*

- Based on code structure and logic
- Tests internal paths, branches, and conditions
- Requires knowledge of implementation

Typical questions:

- Have all decision paths been exercised?
- Are boundary conditions handled correctly?
- Is this logic robust to change?

Functional vs Non-Functional Testing

What dimension of behavior we are evaluating?

Functional Testing

Focus: *What the system does*

- Verifies features and behaviors against requirements
- Checks that inputs produce correct outputs
- Ensures business rules are enforced

Examples:

- Calculating a trade correctly
- Rejecting invalid transactions
- Enforcing account limits

Non-Functional Testing

Focus: *How well the system does it*

- Evaluates quality attributes of the system
- Behavior under realistic and adverse conditions
- Often the source of large-scale failures

Examples:

- Performance and latency
- Reliability and availability
- Security and access control
- Usability and error recovery
- Scalability under load

A system can be functionally correct and still be unsafe to deploy.

Logical Organization of Testing

Different tests focus on **different parts of the system**, at different levels of abstraction.

Unit Testing

Focus: Individual components or classes

- Tests small, isolated pieces of logic
- Fast feedback
- Usually written by developers

Primary risks addressed:

- Local logic errors
- Incorrect assumptions in code

Integration Testing

Focus: Interactions between components

- Verifies contracts between modules or services
- Catches mismatches in data, timing, and assumptions

Primary risks addressed:

- “It worked in isolation”
- Interface and configuration errors

System Testing

Focus: The system as a whole

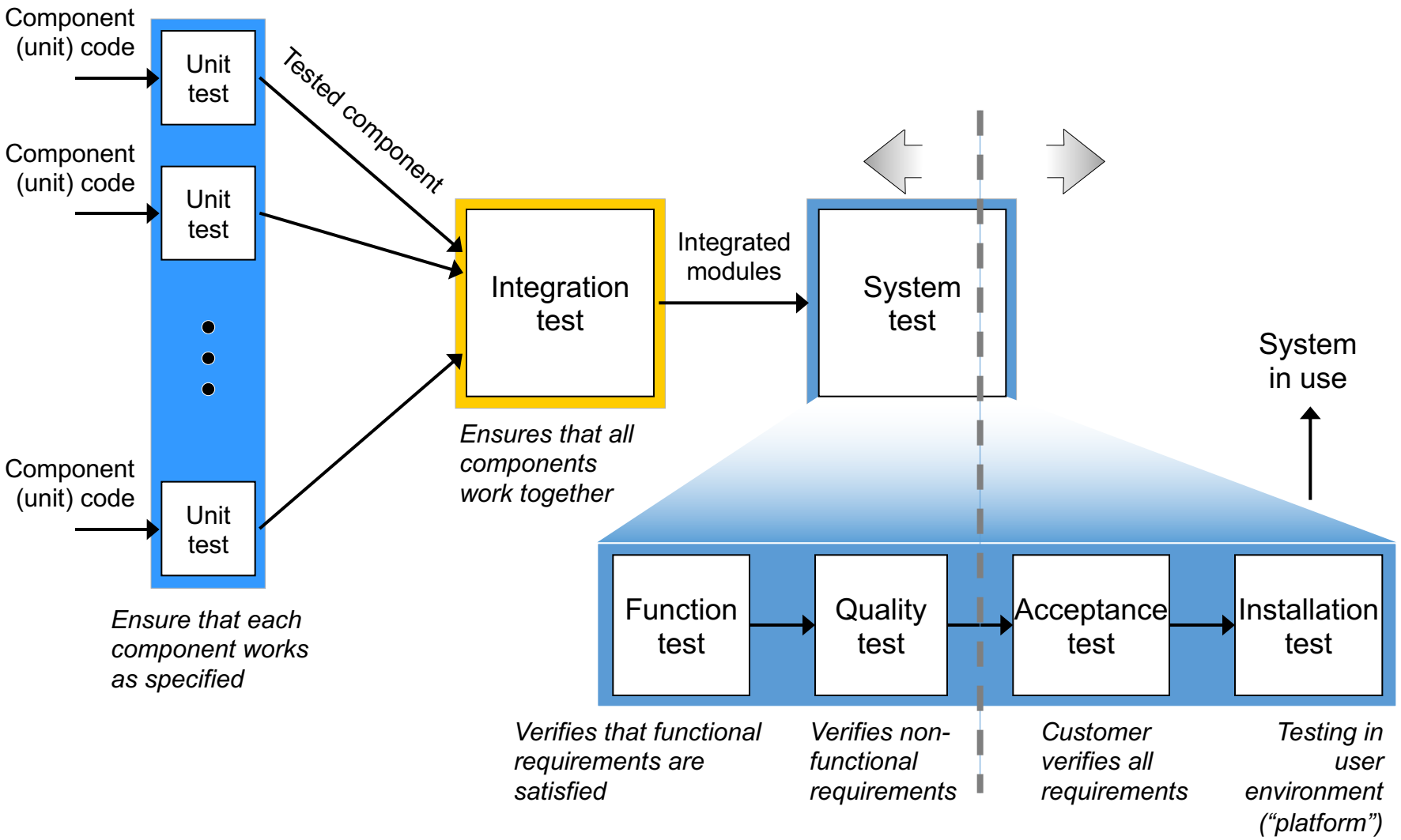
- End-to-end workflows
- Cross-cutting concerns (security, performance, failure behavior)

Primary risks addressed:

- System-level failures
- Emergent behavior

Logical Organization of Testing

(Usually not done in a linear step-by-step order and completed when the last step is reached!)

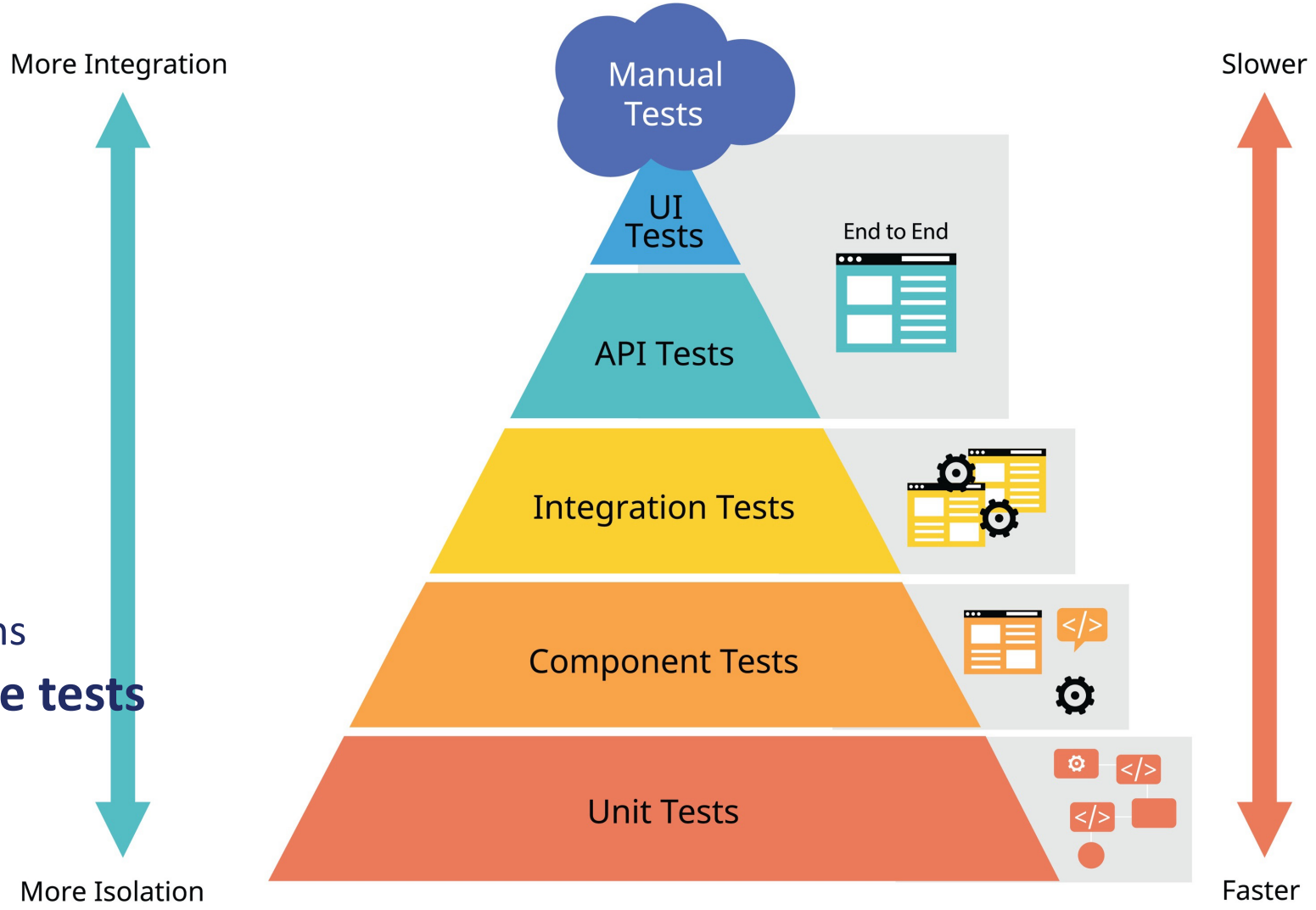


Test Pyramid

Many unit tests
- Fast
- Cheap to run
- Easy to debug

Fewer integration tests
- Slower
- More complex
- Validate component interactions

Even fewer system / acceptance tests
- Slowest
- Most expensive
- Hardest to diagnose



Test Artifacts

Different tests focus on **different parts of the system**, at different levels of abstraction.

Making Testing Decisions Explicit

Testing produces **artifacts** that document:

- what was tested
- why it was tested
- what was found
- what remains uncertain

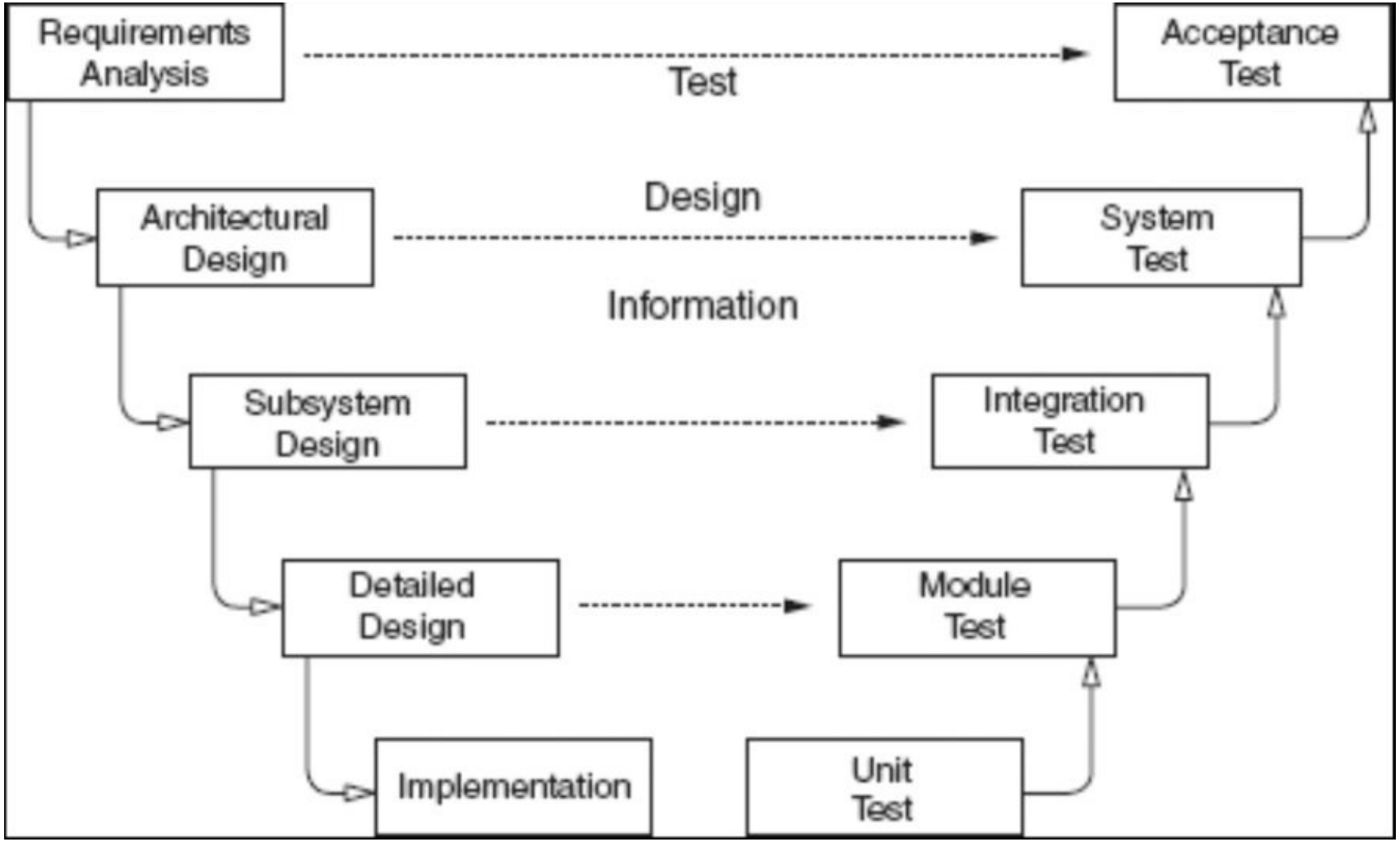
Why Artifacts Matter

- Support **review and accountability**
- Enable **regression testing**
- Provide **evidence** for audits and compliance
- Preserve institutional knowledge as systems evolve

Common Test Artifacts

- **Test Plans**
  - Scope, goals, and priorities
- **Test Cases**
  - Inputs, conditions, expected outcomes
- **Bug / Defect Reports**
  - Observed failures and reproduction steps
- **Test Metrics**
  - Coverage, pass/fail rates, trends

Another View: The "V Model"



Code Coverage: What It Measures

Which parts of the code were executed by tests?

Common Coverage Metrics

- **Statement coverage** — executed lines
- **Branch coverage** — executed control paths
- **Path coverage** — executed execution paths *(theoretical)*

What Coverage Does *Not* Measure

- Correctness of behavior
- Quality of assertions
- Missing requirements or assumptions

Coverage: Necessary but Insufficient

Coverage Is a Baseline, Not a Goal

- **0% coverage** → no evidence
- **Low coverage** → large blind spots
- **High coverage** → still possible failures

Why Coverage Alone Is Not Enough

- Tests may assert the wrong behavior
- Important scenarios may be missing
- Rare or systemic failures may never execute

Coverage Example: Trader Bonus

```
def calculate_trader_bonus(score, years, is_lead):
    bonus = 0                          # S1

    if score > 50 and years > 10:      # S2
        bonus += 5000                  # S3
    elif score > 80 or years > 7:      # S4
        bonus += 3000                  # S5
    else:
        bonus += 1000                  # S6

    if years > 5:                      # S7
        bonus *= 1.2                   # S8

    if is_lead:                        # S9
        bonus += 10000                 # S10

    return bonus                       # S11
```

Statement coverage

% of executed lines

Unit Test:

Arrange: score = 0, years = 0, is_lead=False
Act: execute calculate_trader
Assert:

1. What should be the assertion?
2. What statements (S1,S2,…) are executed?

Coverage Example: Trader Bonus

```
def calculate_trader_bonus(score, years, is_lead):
    bonus = 0                          # S1

    if score > 50 and years > 10:      # S2
        bonus += 5000                  # S3
    elif score > 80 or years > 7:      # S4
        bonus += 3000                  # S5
    else:
        bonus += 1000                  # S6

    if years > 5:                      # S7
        bonus *= 1.2                   # S8

    if is_lead:                        # S9
        bonus += 10000                 # S10

    return bonus                       # S11
```

Statement coverage

% of executed lines

Unit Test:

Arrange: score = 0, years = 0, is_lead=False
Act: execute calculate_trader
Assert: result == 1000

1. What should be the assertion?
   result == 1000
2. What statements (S1,S2,…) are executed?
   S1,S2,S4,S6,S7,S9,S11

Statement coverage: 7 of 11 statements executed, 63.6%

Coverage Example: Trader Bonus

```
def calculate_trader_bonus(score, years, is_lead):
    bonus = 0                          # S1

    if score > 50 and years > 10:      # S2
        bonus += 5000                  # S3
    elif score > 80 or years > 7:      # S4
        bonus += 3000                  # S5
    else:
        bonus += 1000                  # S6

    if years > 5:                      # S7
        bonus *= 1.2                   # S8

    if is_lead:                        # S9
        bonus += 10000                 # S10

    return bonus                       # S11
```

Statement coverage

% of executed lines

Unit Test:

Arrange: score = 0, years = 0, is_lead=False
Act: execute calculate_trader
Assert: result == 1000

Coverage: S1, S2, S4, S6, S7, S9, S11
7/11 = 63.6%

calculate_trader_bonus(95,11,True)
S1, S2, **S3**, S7, **S8**, S9, **S10**, S11
total coverage: 10/11

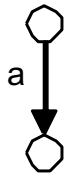
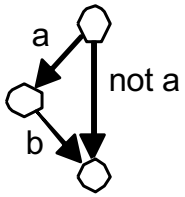
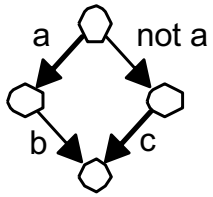
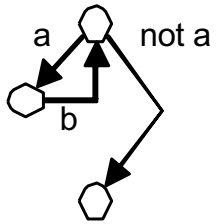
calculate_trader_bonus(85, 6, False)
S1, S2, S4, **S5**, S7, S8, S9
Total coverage: 11/11

Coverage – Control-Flow Graph

Branch / Edge coverage: Every edge (branch) of the control flow is traversed at least once by some test case

Path coverage: All distinct paths through the program to be traversed at least once

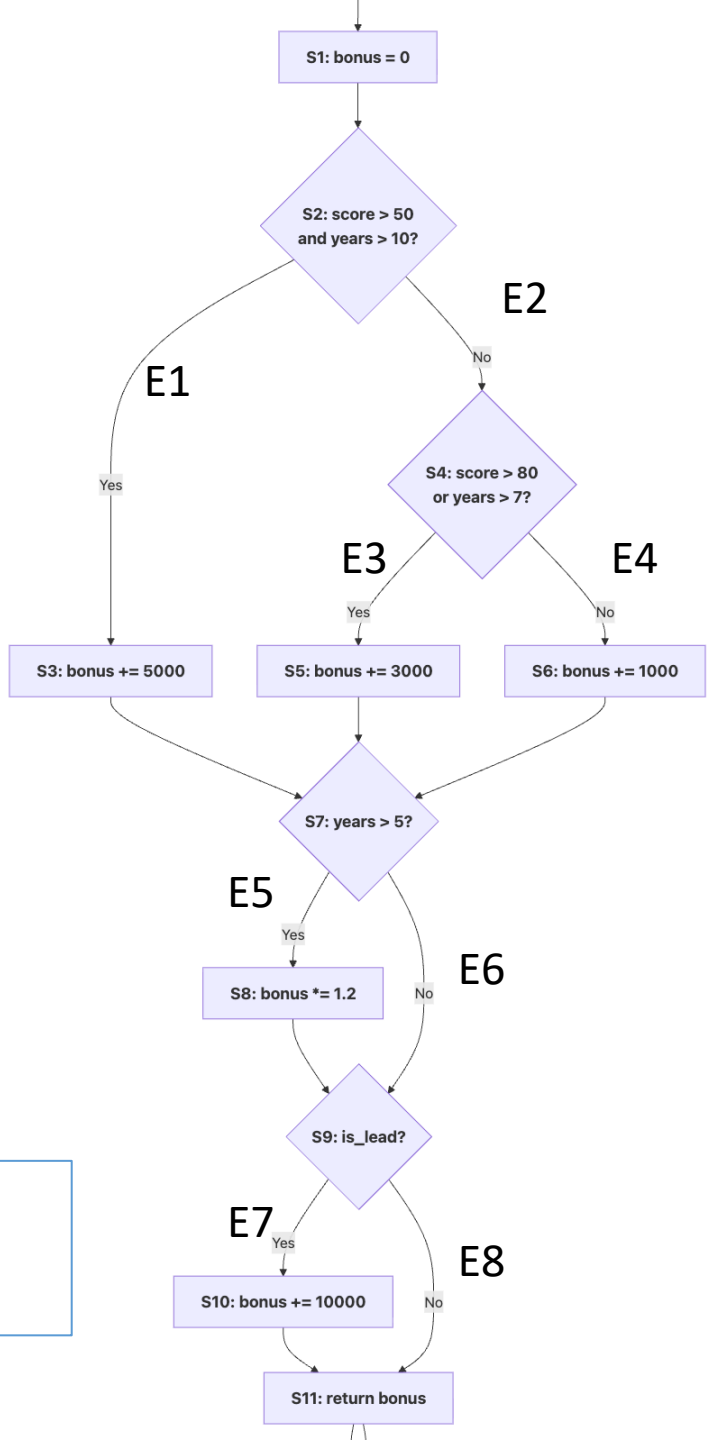
Requires creating a control-flow graph of the function.

a;	a; b;	*if* a *then* b;	*if* a *then* b *else* c;	*while* a *do* b;
				

Branch Coverage: Trader Bonus

```python
def calculate_trader_bonus(score, years, is_lead):
    bonus = 0                          # S1

    if score > 50 and years > 10:      # S2
        bonus += 5000                  # S3
    elif score > 80 or years > 7:      # S4
        bonus += 3000                  # S5
    else:
        bonus += 1000                  # S6

    if years > 5:                      # S7
        bonus *= 1.2                   # S8

    if is_lead:                        # S9
        bonus += 10000                 # S10

    return bonus                       # S11
```

```
calculate_trader_bonus(100, 10, True): E1, E5, E7      → 3/8
calculate_trader_bonus(100, 4, False): E2, E3, E6, E8 → 7/8
calculate_trader_bonus(50, 4, False):  E2, E4, E6, E8 → 8/8
```



Path Coverage: Trader Bonus

```python
def calculate_trader_bonus(score, years, is_lead):
    bonus = 0                          # S1

    if score > 50 and years > 10:      # S2
        bonus += 5000                  # S3
    elif score > 80 or years > 7:      # S4
        bonus += 3000                  # S5
    else:
        bonus += 1000                  # S6

    if years > 5:                      # S7
        bonus *= 1.2                   # S8

    if is_lead:                        # S9
        bonus += 10000                 # S10

    return bonus                       # S11
```

Distinct Paths: (10 possible)

E1, E5, E7 (100, 11, True)

E1, E5, E8 (100, 11, False)

~~E1, E6, E7~~ impossible

~~E1, E6, E8~~ impossible

E2, E3, E5, E7 (85, 7, True)

E2, E3, E5, E8 (85, 7, False)

E2, E3, E6, E7 (85, 5, True)

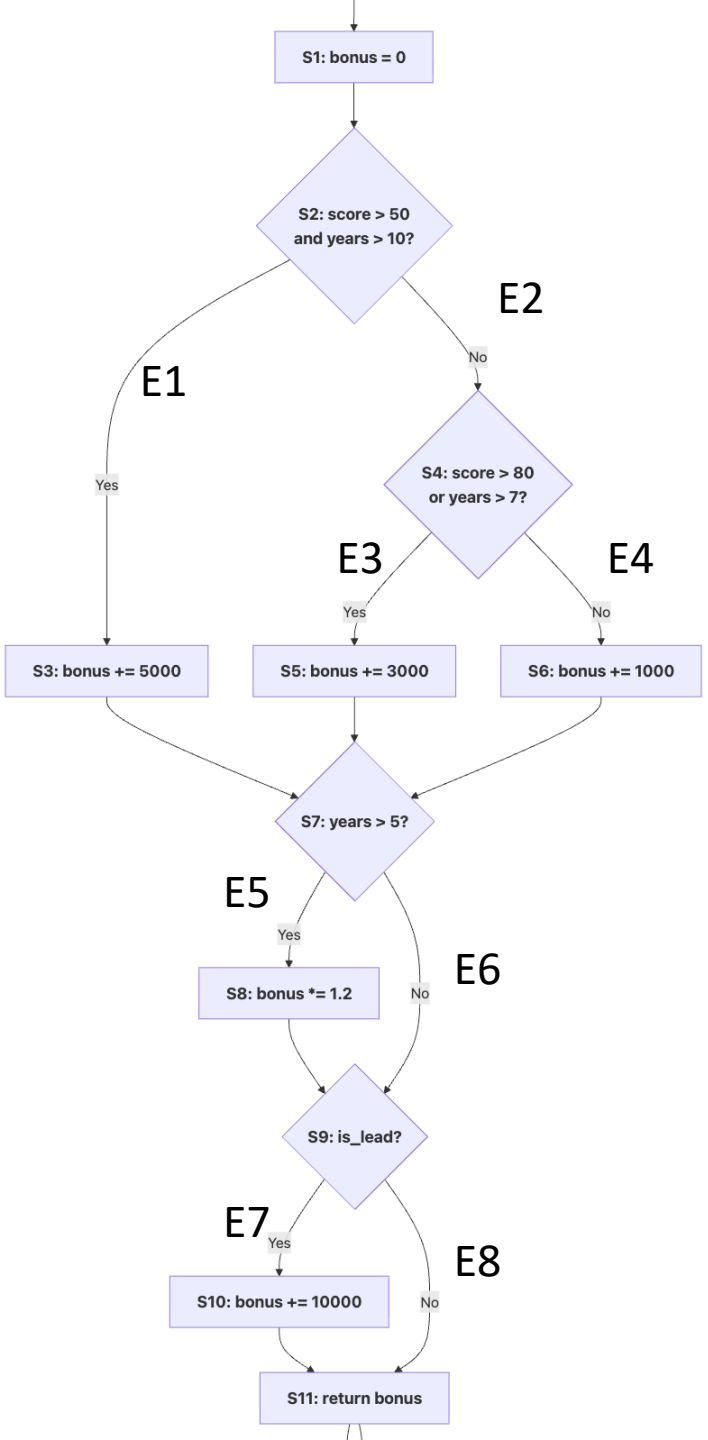
E2, E3, E6, E8 (85, 5, False)

E2, E4, E5, E7 (79, 7, True)

E2, E4, E5, E8 (79, 7, False)

E2, E4, E6, E7 (79, 5, True)

E2, E4, E6, E8 (79, 6, False)



What We Still Might Have Missed

Even with full statement + branch coverage, we may not have tested:

- Boundary values: score=50, score=80, years=5, years=7, years=10
- Short-circuit subtleties (did both sides of AND/OR matter independently?)
- Numeric precision issues (* 1.2 in floating point / rounding rules)
- Policy intent questions (is it correct that years=8 qualifies in the elif even if score is mediocre?)

Coverage tells us **where execution happened**.
It does not validate that the **policy** is correct or complete.

Coverage is a quantity metric, **not a quality metric**

Coverage

```python
def calculate_price(stock_price, num_items, discount):
    quantity_discount = 0.0
    result = 0.0

    if num_items >= 10:
        quantity_discount = 15;
    elif num_items >= 5:
        quantity_discount = 10;
    else:
        quantity_discount = 0

    if discount > quantity_discount:
        quantity_discount = discount

    result = stock_price/100.0 * (100-quantity_discount) * num_items;

    return result;

calculate_price(0,5,10)
calculate_price(0,10,10)
#calculate_price(0,1,10)
```

Coverage for **price.py**: 80%

14 statements | 12 run | 2 missing | 0 excluded | 2 partial

« prev ^ index » next coverage.py v7.4.1, created at 2024-01-29 13:14 -0500

```python
1  def calculate_price(stock_price, num_items, discount):
2      quantity_discount = 0.0
3      result = 0.0
4
5      if num_items >= 10:
6          quantity_discount = 15;
7      elif num_items >= 5:
8          quantity_discount = 10;
9      else:
10         quantity_discount = 0
11
12     if discount > quantity_discount:
13         quantity_discount = discount
14
15     result = stock_price/100.0 * (100-quantity_discount) * num_items;
16
17     return result;
18
19 calculate_price(0,5,10)
20 calculate_price(0,10,10)
21 #calculate_price(0,1,10)
```

https://coverage.readthedocs.io/

```
% coverage run --branch price.py
% coverage report
Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
price.py       14      2      6      2    80%   10, 13
-----------------------------------------------------------
TOTAL          14      2      6      2    80%
```

Duke

Input-Space Coverage

Choosing the Right Tests When You Can’t Test Everything

Duke

The Input Space Problem

The Core Problem

- Software systems accept **far more inputs than we can test**
- Exhaustive testing is infeasible, even for simple programs

Key Constraint

- **Testing can only evaluate a finite set of inputs.**

Implication

- Input selection is an **engineering decision**
- Some inputs must be treated as **more important than others**
  - Representative
  - More likely to expose failures

Black-Box Testing as Input-Space Reasoning

Black-Box Testing Focuses on Inputs and Outputs

- Tests the system through its **external interface**
- Internal implementation details are ignored
- Behavior is evaluated based on **observable results**

Input-Space Perspective

- The system defines an **input domain**
- Different inputs may trigger different behaviors
- Testing is about selecting **meaningful input values**

Key Point

Input-space coverage complements structural coverage.
One looks at *values*, the other at *code*.

Equivalence Class Testing

Equivalence class testing groups inputs that are expected to behave the same way

Equivalence Classes

- Set of inputs that the system should treat equivalently
- If one value in the class works, others are *assumed* to behave similarly

Testing Strategy

- Identify equivalence classes
- Select **one representative value** from each class
- Test each representative

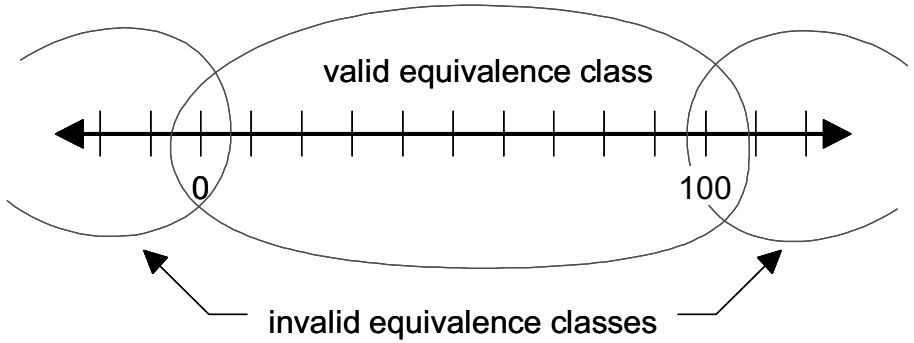
Key Assumption

Inputs in the same equivalence class produce the same behavior.

Equivalence Classes

Range condition:

- below the range (invalid)
- in the range (valid)
- above the range (invalid).



Given input: valid input and invalid inputs.

Membership within a collection:
member exists or the member does not exist

Boolean values: True and False.

Valid Equivalence Classes: Expected to produce normal system behavior

Invalid Equivalence Classes: Must trigger defined failure behavior

Boundary Value Analysis

Why Boundaries Matter

- Many failures occur **at the edges of allowed input ranges**
- Off-by-one errors are common, < rather than <=
- Limits and thresholds are frequent sources of bugs

Use boundary values for test cases. Create test cases -

- just below the boundary
- on the boundary
- just above the boundary.

Example: Trade Amount Validation

Assume a trading system accepts a **trade amount** with the following rule:

Trade amount must be **between $1,000 and $1,000,000 (inclusive)**

Equivalence Classes

Valid

- $1,000 – $1,000,000

Invalid

- Less than $1,000
- Greater than $1,000,000
- Non-numeric values
- Missing value

Boundary Values

Category	Example Values
Just below minimum	$999
Minimum (valid)	$1,000
Just above minimum	$1,001
Just below maximum	$999,999
Maximum (valid)	$1,000,000
Just above maximum	$1,000,001

What Input-Space Coverage Still Misses

Interactions between inputs

- Multiple fields influencing each other

State-dependent behavior

- Results depend on prior actions or history

Temporal effects

- Order and timing of operations matter

Emergent system behavior

- Issues only appear under load or failure conditions

Exercise: Design Test Cases for Trade Settlement

Given this requirement:

Settlement amount must be between $100 and $10,000,000.

Premium accounts get 2% discount on amounts over $100,000.

Tasks:

1. Identify equivalence classes
2. Identify boundary values
3. Propose test cases
4. What statement/branch coverage would you expect?
5. Which test cases matter most? Why?

Integration, System, Acceptance, and Non-Functional Testing

Testing Behavior Across Components, Boundaries, and Stakeholders

Duke

Why Integration and System Testing Matter

Failures Happen Between Components

- Individual components can behave correctly in isolation
- Failures often occur when components **interact**
- System behavior emerges from **coordination, timing, and assumptions**

"It Worked in Isolation" Is a Common Failure Mode

- Mismatched data formats
- Incorrect assumptions about timing or ordering
- Partial failures that were never considered
- Missing or inconsistent error handling

FinTech Example: Trade Execution Pipeline

A simplified trade execution flow may involve:

- Order validation
- Risk checks
- Pricing
- Execution
- Settlement
- Reporting

Each step may be correct on its own —
but failures often appear **at the boundaries between them**.

Integration and system testing focus on interaction risk, not local correctness.

Integration Testing

Evaluates how **multiple components work together**.

What Integration Tests Look For

- Correct data exchange between components
- Consistent interpretation of inputs and outputs
- Proper handling of partial failures
- Timing and ordering assumptions

Trade Execution Pipeline Example

- Order validation passes correct data to risk checks
- Risk results are interpreted correctly by pricing
- Pricing outputs are compatible with execution systems
- Execution results are correctly forwarded to settlement

Integration Testing Strategies

- **Bottom-Up**
  - Start with low-level components
  - Gradually integrate higher-level logic
- **Top-Down**
  - Start with high-level workflows
  - Stub or mock lower-level components
- **Sandwich (Hybrid)**
  - Combine top-down and bottom-up approaches
- **Vertical Slicing**
  - Test a complete path through the system early

Trade Execution Pipeline Example

- Bottom-up: validate settlement and reporting correctness
- Top-down: validate user-visible trade flows
- Vertical slice: validate one complete trade path end-to-end

System Testing

Evaluates the **entire system end-to-end** in a realistic environment.

- Focuses on complete workflows
- Includes multiple components and services
- Observes externally visible behavior

What System Tests Validate

- End-to-end correctness
- Cross-cutting concerns:
  - security
  - performance
  - reliability
- Behavior under realistic conditions

System Testing: Trade Execution Pipeline

A system test may validate:

- A trade is submitted, priced, executed, settled, and reported correctly
- Failures at any stage are handled safely
- No partial or inconsistent state is exposed

At this level, we are no longer asking: “Did each part work?”

We are asking: “Did the system behave acceptably?”

Acceptance Testing

Evaluates whether the system **meets agreed-upon expectations** from the perspective of its users or stakeholders.

What Acceptance Tests Focus On

- End-to-end behavior
- User-visible outcomes
- Business rules and constraints
- Explicit success and failure conditions

Why Acceptance Testing Matters

- Makes expectations explicit
- Aligns engineers and stakeholders
- Acts as a contract for release decisions
- Provides evidence of fitness for use

Key Characteristics

- Black-box in nature
- Based on requirements, policies, or contracts
- Often defined collaboratively
- Stable across implementation changes

Non-Functional Testing at the System Level

We must evaluate how the system behaves under real conditions.

- Performance and latency under load
- Reliability and availability
- Security and access control
- Usability and operational clarity

Trade execution example:

- System remains responsive during market spikes
- Failures do not expose inconsistent state
- Audits and logs are complete and reliable

Professional Practice & Testing Maturity

How Testing Actually Works on Real Teams

Duke

Unit Testing in Practice

What Is a “Unit”?

A **unit** is the *smallest piece of behavior that can be tested meaningfully in isolation*.

- Often a class or function
- Sometimes a small group of closely related functions
- Defined by **responsibility**, not file boundaries

Purpose of Unit Tests

- Verify **local correctness**
- Enforce invariants and assumptions
- Provide fast feedback during development
- Support safe refactoring

Who Owns Unit Tests?

- Unit tests are written and maintained by **developers**
- They are part of the codebase, not external documentation
- A feature is not “done” without its unit tests

Unit tests protect design decisions from silent breakage.

Test “FIRST” Principles

- **Fast** – sub-second execution, so they can be run frequently
- **Independent** – tests shouldn’t depend upon one another or shared state
- **Repeatable** -Tests should always provide the same result.
- **Self-checking** – Embed the expected value(s), so no one needs to read after the test is run
- **Timely** – Tests should be written early, alongside or before code

Test Smells

- **Brittle tests:** Break when unrelated code changes
- **Obscure intent:** Test names or logic do not explain *what is being tested*
- **Logic in tests:** Complex conditionals or loops inside tests
- **Duplicated setup:** Repeated, verbose test configuration
- **Slow tests:** Tests that discourage frequent execution

Why Test Smells Matter

- They reduce trust in the test suite
- They increase maintenance cost
- They hide real failures among false alarms

Regression Testing

Checks that **existing behavior has not been broken** by changes.

Why Regressions Happen

- Hidden dependencies
- Incomplete understanding of existing behavior
- Changes that violate prior assumptions
- Interactions between components

How Regression Testing Works

- Existing tests are re-run after changes
- Failures signal unintended side effects
- Tests act as **guards on past decisions**

Beizer's Five Levels of Testing Maturity

Level 0 — Testing = Debugging

- Testing happens only after something breaks.
- Reactive and incomplete

Level 1 — Testing Proves Correctness

- If tests pass, the software must be right.
- Tests check correctness against specifications

Level 2 — Testing Finds Bugs

- The goal is to make the software fail.

Level 3 — Testing Reduces Risk

- Testing provides evidence where failure would matter most.

Level 4 — Testing Is a Design Discipline

- Testing shapes how systems are structured and decisions are made.
- Quality is engineered, not inspected in

Using Generative AI for Testing

Brainstorming test cases

- Identify edge cases and missing scenarios

Drafting test skeletons

- Create initial test structure for review

Generating input variations

- Especially useful for validation and boundary cases

Summarizing existing tests

- Help understand large or unfamiliar test suites

Using Generative AI for Testing: Cautions

- Treat AI output as a **draft**, not an answer
- Review and refine generated tests
- Ask AI *why* a test case exists
- Cross-check against requirements and specifications
- **AI is a collaborator, not an authority.**

Where AI Is Dangerous

- Confident but incorrect assertions
- Missing critical edge cases
- Reinforcing flawed assumptions
- Generating tests that pass but test the *wrong thing*

Production Data in Test

What are the implications of production data in test?

Using **production data in a test environment** has several implications, both positive and negative. Here's a breakdown:

Implications of Using Production Data in Testing

Category	Implications
Security & Compliance	Risk of exposing sensitive or personally identifiable information (PII), violating regulations like GDPR, HIPAA, or CCPA.
Data Integrity	Changes made during testing can corrupt or alter real data if not properly segregated.
Performance Impact	Large datasets can slow down testing, requiring more resources than necessary.
Realism & Accuracy	Testing with real-world data improves accuracy by revealing edge cases and unexpected behaviors.
Debugging Complexity	Production data can introduce noise, making it harder to isolate specific test cases.
Cost Considerations	Storage and processing of full-scale production data can increase costs.
Test Automation Challenges	Production data may be dynamic, causing flaky or inconsistent test results.
Masking & Anonymization	If production data is used, it should be anonymized or masked to reduce risks.

Best Practices

- **Use Synthetic or Sanitized Data** whenever possible.
- **Anonymize/Mask Sensitive Information** before using production data in testing.
- **Ensure Proper Access Controls** to prevent unauthorized exposure.
- **Create a Dedicated Test Environment** separate from production.
- **Automate Data Refresh and Cleanup** to maintain test data consistency.

Would you like guidance on implementing data masking or test data generation strategies?

Key Takeaways

Testing is not a phase, a tool, or a checklist.
It is a **disciplined way of managing risk and uncertainty** in software systems.

Key Lessons

- Testing provides **evidence**, not proof
- Different tests address **different risks**
- Coverage measures execution, not correctness
- Input selection is an engineering decision
- Integration and system testing expose emergent failures
- Tests protect decisions as systems evolve